



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: VELÁZQUEZ CORTÉS CRISTINA
DEL CARMEN

N° de Cuenta: 320005864

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 26/09/2026

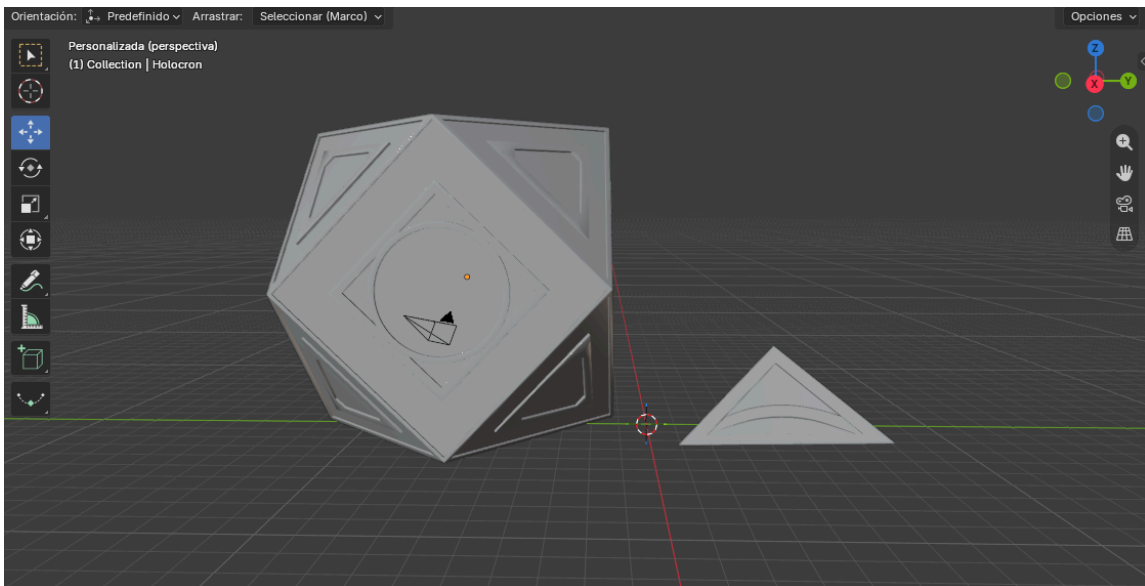
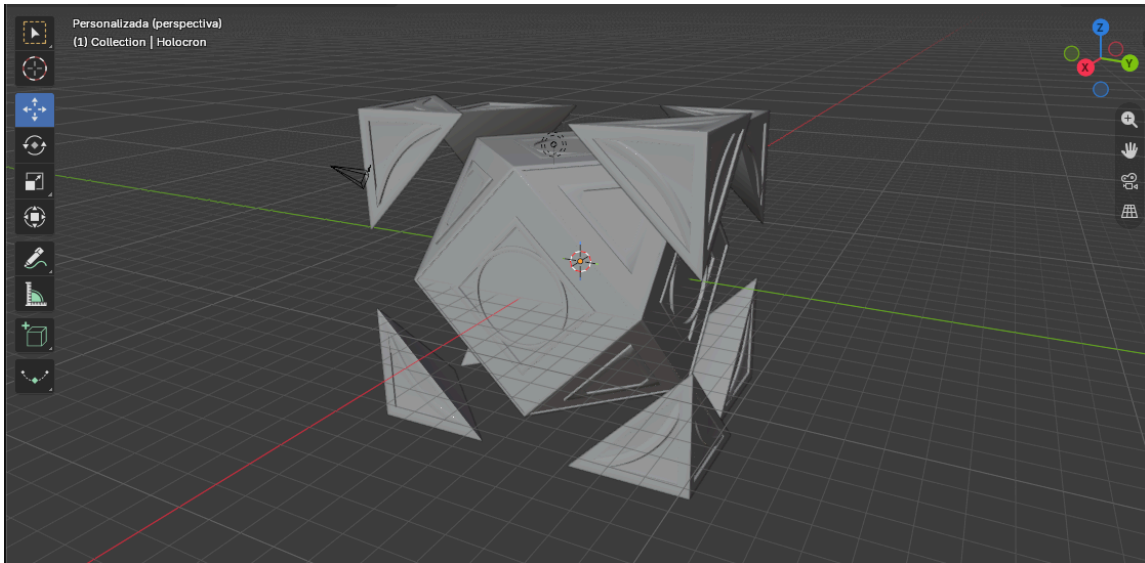
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Importar el modelo del Holocron con jerarquía

Comenzamos separando el modelo del holocron proporcionado por el profesor en Blender, separando las esquinas del centro.



Para la práctica, decidí usar un único modelo para las esquinas y rotarlo en código para cada esquina. Importamos los modelos:

```
Model baseholocron;  
Model esquina;
```

```
baseholocron = Model();
baseholocron.LoadModel("Models/baseholocron.obj");
esquina = Model();
esquina.LoadModel("Models/esquina.obj");
```

2.- Con teclado de forma independiente poder girar cada esquina alrededor del centro del Holocrón.

La base del Holocrón se colocó primero en la escena mediante una traslación sobre el eje Y para que no se corte con el piso,. Posteriormente, su matriz de transformación fue almacenada en **modelaux**. Esta matriz funciona como referencia para las demás piezas, permitiendo establecer la relación jerárquica entre el cuerpo central y las esquinas.

```
//Base Holocron
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 3.5f, 0.0f));
modelaux = model;
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
baseholocron.RenderModel();

//Esquina +X, +Y, +Z
model = modelaux;
model = glm::translate(model, glm::vec3(-3.47f, 3.47f, 3.42f));
color = glm::vec3(1.0f, 0.0f, 1.0f);
model = glm::rotate(model, -1.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 43.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 55.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquina.RenderModel();

//Esquina -X, +Y, +Z
model = modelaux;
model = glm::translate(model, glm::vec3(-3.43f, 3.47f, -3.42f));
color = glm::vec3(0.5f, 0.0f, 0.5f);
model = glm::rotate(model, -1.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, -46.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 55.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquina.RenderModel();
```

```
//Esquina +X, -Y, +Z
model = modelaux;
model = glm::translate(model, glm::vec3(-3.43f, -3.47f, -3.42f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::rotate(model, -91.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, -46.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 55.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion12()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquina.RenderModel();
```

```
//Esquina -X, -Y, +Z
model = modelaux;
model = glm::translate(model, glm::vec3(-3.47f, -3.47f, 3.42f));
color = glm::vec3(1.0f, 1.0f, 0.0f);
model = glm::rotate(model, 91.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 43.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 54.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion13()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquina.RenderModel();
```

```
//Esquina +X, +Y, -Z
model = modelaux;
model = glm::translate(model, glm::vec3(3.47f, 3.47f, 3.42f));
color = glm::vec3(0.0f, 1.0f, 1.0f);
model = glm::rotate(model, 181.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 313.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 55.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion14()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquina.RenderModel();
```

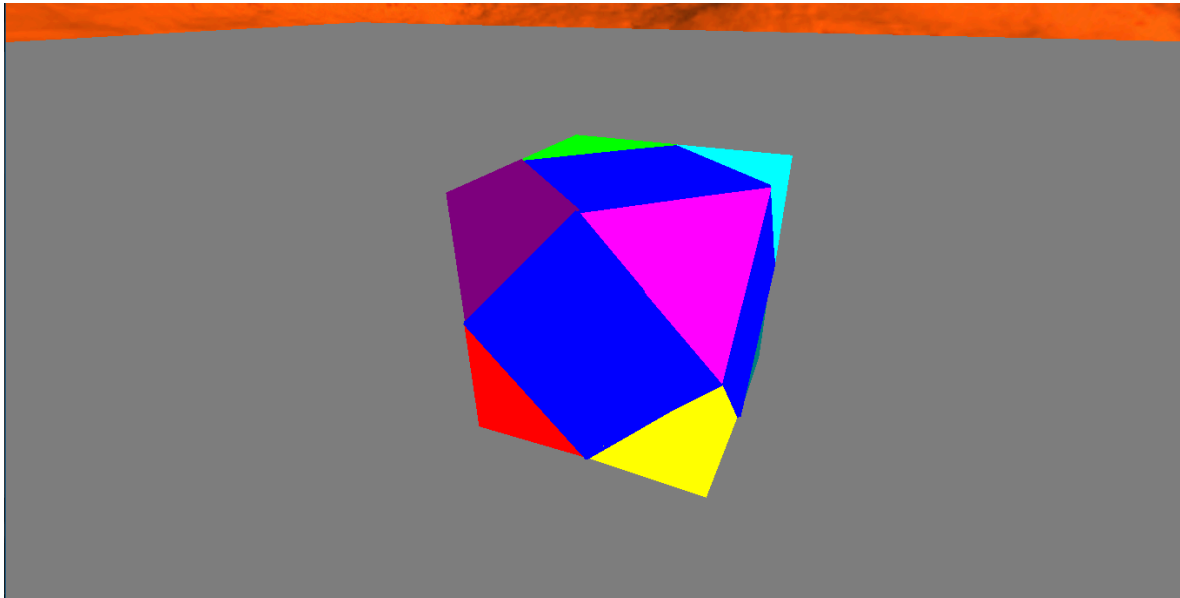
```
//Esquina -X, +Y, -Z
model = modelaux;
model = glm::translate(model, glm::vec3(3.43f, 3.47f, -3.42f));
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::rotate(model, 181.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 43.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 55.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion15()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquina.RenderModel();
```

```
//Esquina +X, -Y, -Z
model = modelaux;
model = glm::translate(model, glm::vec3(3.47f, -3.47f, 3.42f));
color = glm::vec3(0.0f, 0.5f, 0.5f);
model = glm::rotate(model, -181.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 43.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 56.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion16()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquina.RenderModel();
```

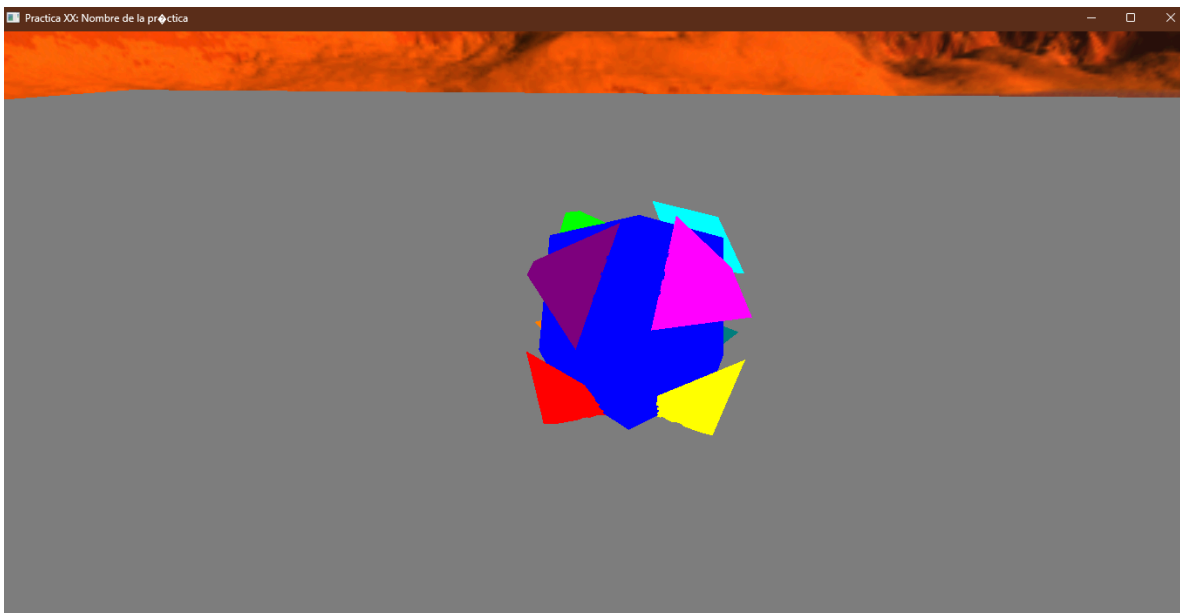
```
//Esquina +X, -Y, -Z
model = modelaux;
model = glm::translate(model, glm::vec3(3.47f, -3.47f, -3.42f));
color = glm::vec3(1.0f, 0.5f, 0.0f);
model = glm::rotate(model, -181.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -43.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 56.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion17()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquina.RenderModel();
```

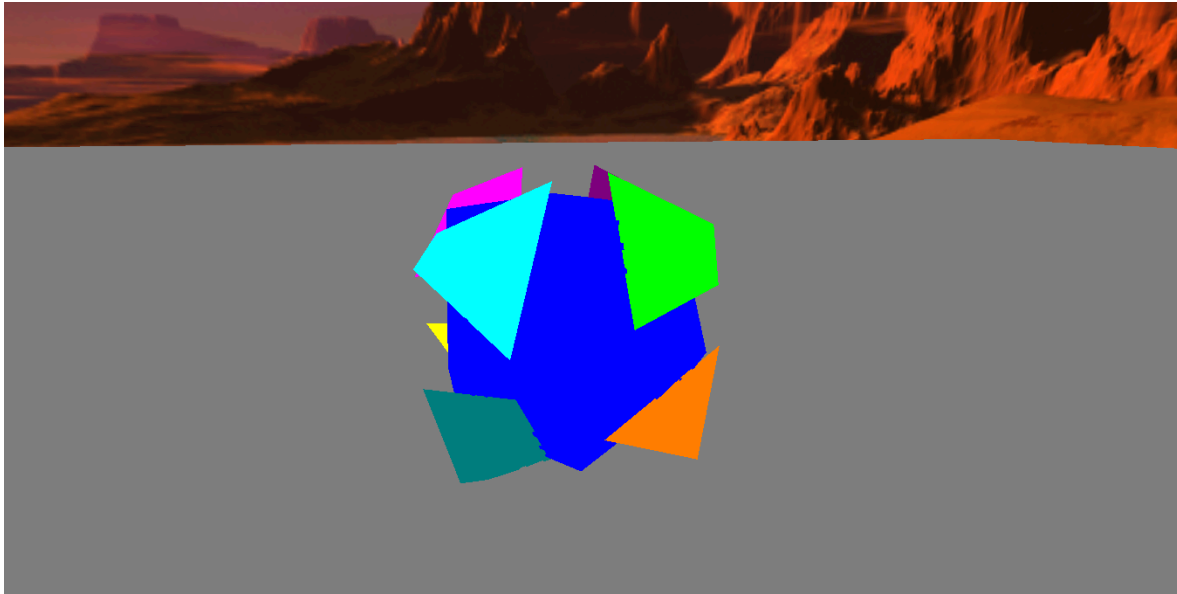
Para construir las ocho esquinas se parte siempre de la transformación almacenada en **modelaux**. Cada instancia del modelo **esquina** recibe una traslación diferente sobre los ejes X, Y y Z para colocarla en uno de los vértices del Holocrón. Después se aplican rotaciones específicas para conseguir que la

misma geometría quede orientada correctamente dependiendo de la posición que ocupa.



Después de colocar correctamente las ocho esquinas se agregó la posibilidad de rotarlas individualmente mediante el teclado. Para esto se utilizaron las variables articulacion10 hasta articulacion17 de la clase Window. Cada variable corresponde a una esquina diferente del Holocrón y su valor se incrementa en pasos de 10 grados cuando se presionan las teclas de los números 1, 2, 3, 4, 5, 6, 7 y 8.

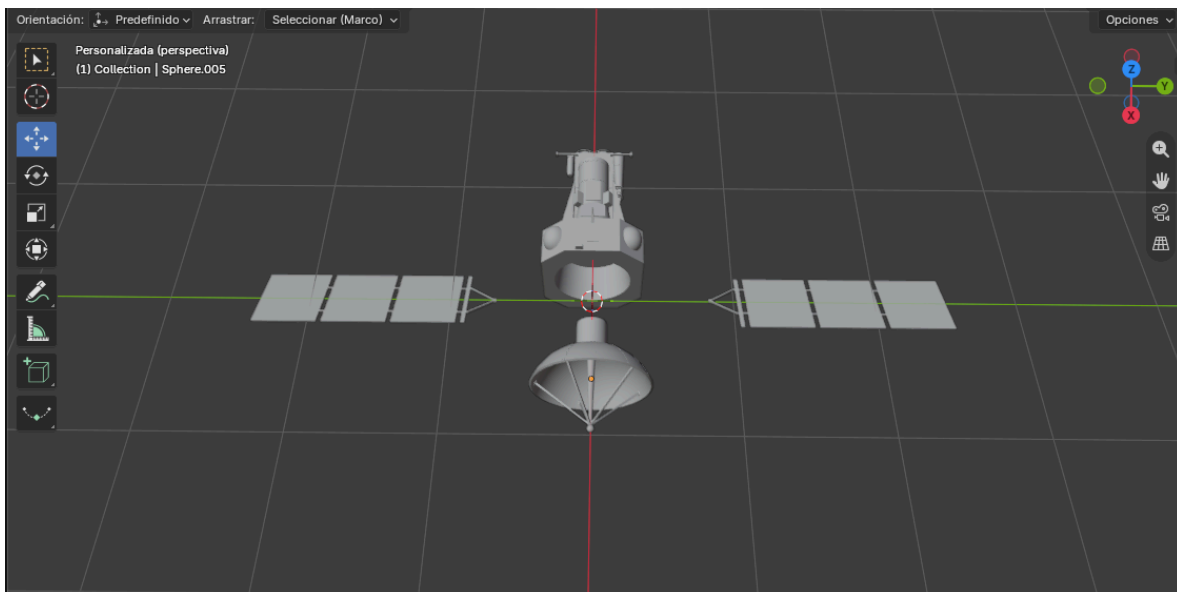




Dentro de **Main.cpp**, después de colocar y orientar inicialmente cada esquina, se aplica una rotación adicional mediante **glm::rotate()**. El ángulo utilizado se obtiene con las funciones correspondientes de la clase Window, mientras que el movimiento se realiza con respecto al eje Y local de cada modelo.

3.- Importar el satélite con jerarquía.

Para el siguiente ejercicio de la práctica se utilizó el satélite adjuntado en el previo, dividido en Blender en cuatro componentes: el cuerpo principal, una antena y dos paneles laterales. Los archivos utilizados fueron cuerposatelite.obj, antena.obj, panel1.obj y panel2.obj.



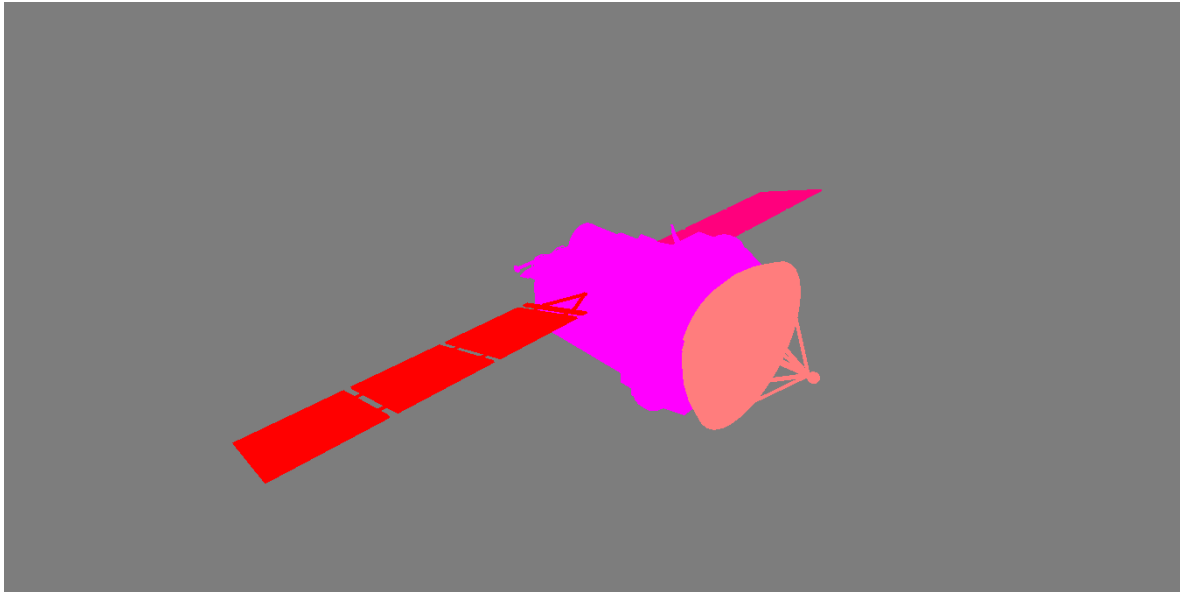
El cuerpo del satélite funciona como elemento principal de la jerarquía. Inicialmente se traslada al lado derecho del holocrón y se guarda la matriz nuevamente en **modelaux**.

```
//Base satellite
color = glm::vec3(1.0f, 0.0f, 1.0f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 3.5f, 20.5f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(7.0f, 7.0f, 7.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cuerposatelite.RenderModel();
```

```
//Antena
model = modelaux;
model = glm::translate(model, glm::vec3(-5.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(7.0f, 7.0f, 7.0f));
color = glm::vec3(1.0f, 0.5f, 0.5f);
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion18()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
antena.RenderModel();
```

```
//Panel derecha
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 6.0f));
model = glm::scale(model, glm::vec3(7.0f, 7.0f, 7.0f));
color = glm::vec3(1.0f, 0.0f, 0.5f);
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion19()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
panel1.RenderModel();
```

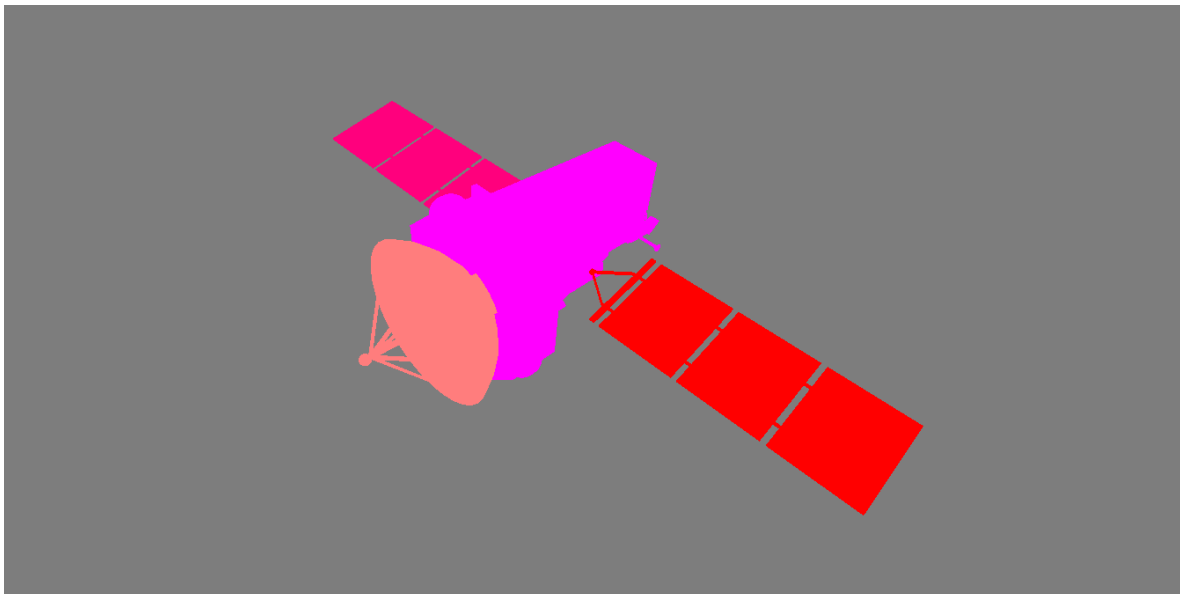
```
//Panel izquierdo
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -6.0f));
model = glm::scale(model, glm::vec3(7.0f, 7.0f, 7.0f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion20()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
panel2.RenderModel();
```

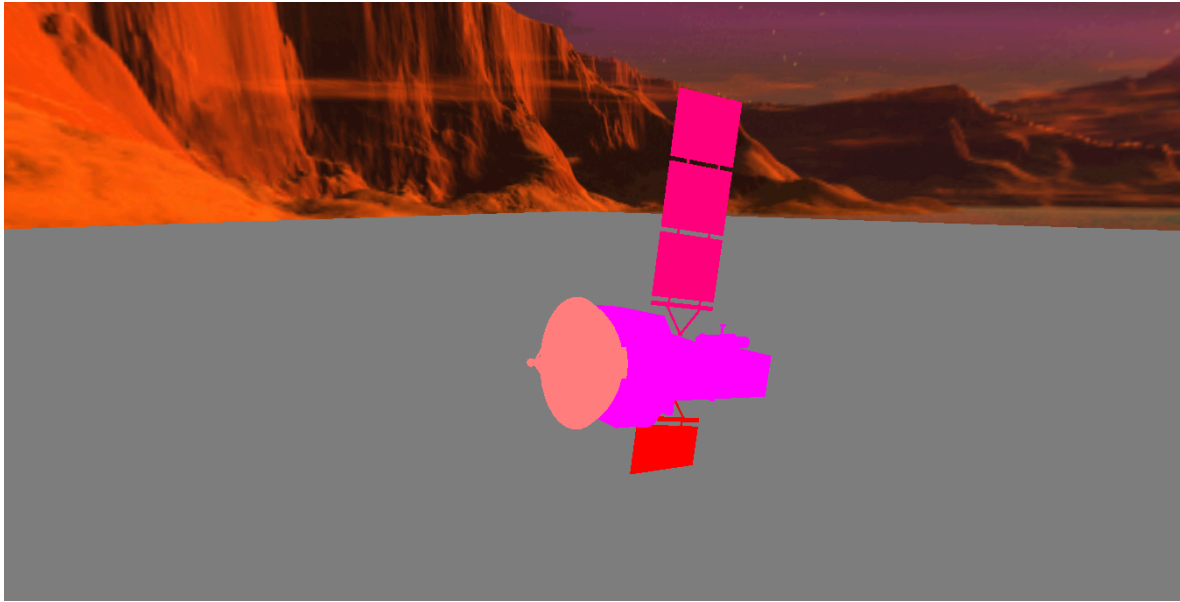


4.- Agregar movimiento al satélite con teclado sobre los ejes X, Y, Z.

Para permitir la interacción con el cuerpo completo del satélite se utilizaron las variables `rotax`, `rotay` y `rotaz`. Estas variables almacenan los ángulos de rotación correspondientes a los tres ejes tridimensionales.

En la función **ManejaTeclado()** de la clase `Window`, la tecla `E` incrementa la rotación sobre el eje `X`, la tecla `R` controla el eje `Y` y la tecla `T` controla el eje `Z`. Cada pulsación incrementa el ángulo en 10 grados.





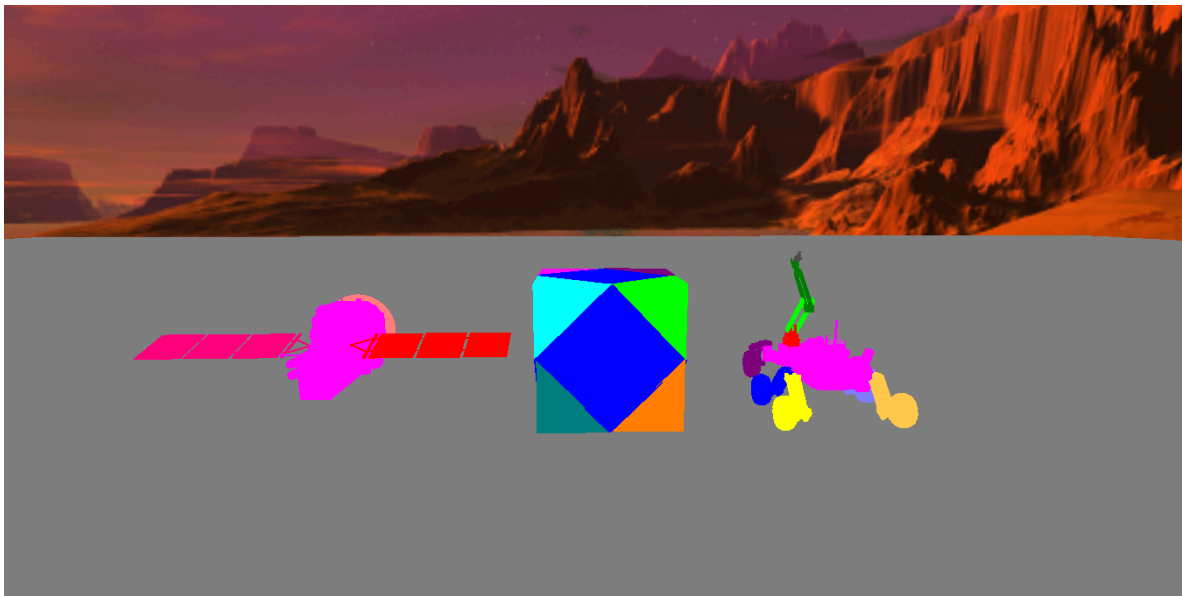
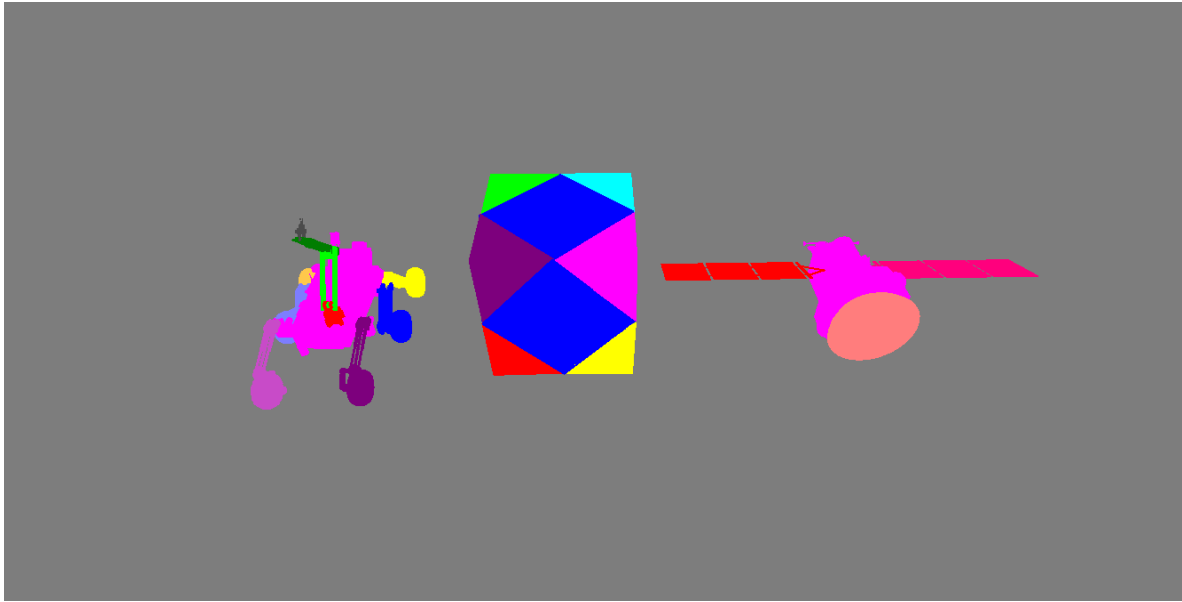
5.- Agregar rotaciones a por lo menos 3 partes del satélite que tengan contexto adecuado. Por ejemplo, si el satélite tiene un panel de celdas solares, solo debería girar en la conexión con el cuerpo, no alrededor de todo el cuerpo.

Finalmente, para los tres elementos secundarios, la antena va a rotar con respecto al eje X, mientras que los paneles se colocan a los costados de la base desplazándose en el eje Z y rotan en el eje Z.

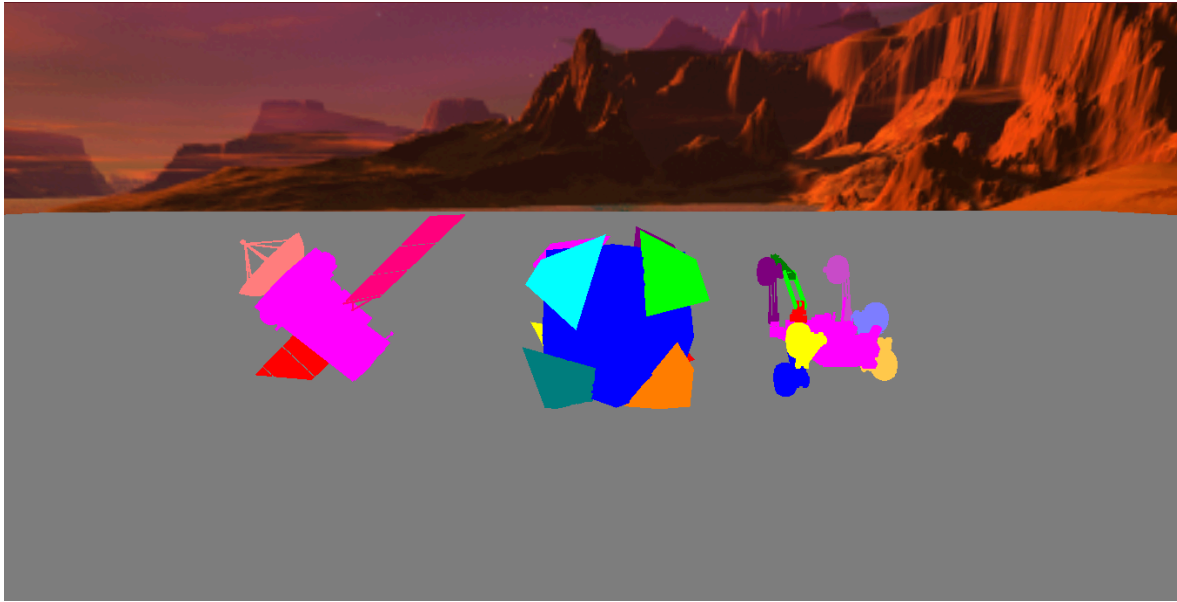


La característica principal de estas transformaciones es que cada pieza conserva la posición de unión con el cuerpo del satélite.

Las 3 figuras mostradas al mismo tiempo:



Moviendo las piezas al mismo tiempo:



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Uno de los problemas que tuve fue con la construcción del Holocrón, para idear cómo colocar las ocho esquinas, pues la rotación no se realizaba de manera correcta. Usé por eso un solo modelo con la cara más grande alineada con el piso para después rotarla según se necesite.

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.
La complejidad del ejercicio fue algo grande por la construcción del holocrón, no hubo explicación o guía de cómo realizarlo.
- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica
Me pareció buena la explicación, aunque fue casi nula la parte de la construcción del holocrón.
- c. Conclusión
La práctica permitió comprender de manera más clara el funcionamiento de las transformaciones jerárquicas dentro de una escena tridimensional. En el caso del Holocrón, la reutilización de un único modelo para las ocho esquinas permitió observar cómo un

mismo objeto puede ser instanciado múltiples veces y adquirir diferentes posiciones y orientaciones mediante matrices de transformación. Mientras que la separación del modelo del satélite me ayudó a comprender mejor las herramientas de blender para separar modelos en partes. Además, el control independiente de cada esquina mediante el teclado permitió aplicar rotaciones locales sin necesidad de modificar la geometría original.